5. 확장 실험

본 연구는 원곡 재현(§3–§4)을 넘어 여러 방향의 확장 실험을 수행하였다. hibari의 **마디 단위 생성** 구현 및 정량 평가는 §6에서 별도로 다룬다.

5.1 다른 곡으로의 일반화 — solari 실험 결과

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice leading	25	0.043	0.078 ± 0.004	0.073
DFT	15	0.071	0.0824 ± 0.0029	0.0773

hibari에서는 DFT가 최우수(0.0213)였지만, solari에서는 frequency/Tonnetz가 동률에 가깝고 DFT는 오히려 악화된다. 12-PC 구조에서는 Tonnetz 지름 한계와 함께 DFT의 분해능도 제한된다($K = 15$). 분해능 제한의 원인: pitch class 12개를 모두 사용하는 곡(solari, aqua)에서는 indicator vector가 $(1, 1, \ldots, 1)$이 되어 DFT 계산 시 $k \neq 0$인 모든 계수가 이론적으로 0 — 12개 단위원 벡터의 합이 상쇄된다. **모든 PC subset이 거의 같은 DFT magnitude 벡터를 가지게 되므로 서로 다른 집합을 DFT 거리로 구별할 여지가 사라진다.** hibari처럼 7개 PC만 쓰는 경우에만 indicator vector가 sparse하여 DFT magnitude 차이가 선명히 유지된다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
JS (이진 OM)	0.106	0.168	0.032
JS (연속값 OM)	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 (continuous 기준) 우위이다.

연속값 OM의 효과 solari에서도 연속값 OM은 이진 OM 대비 개선을 보였다. Transformer 기준 binary JS 0.032 → continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 FC-cont (§4.3, 83.9% 감소)와 같은 방향이다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	DFT	FC	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	Tonnetz (frequency)	Transformer	선율적 진행, 시간 의존

5.2 클래식 대조군 — Bach Fugue 및 Ravel Pavane

곡 기본 정보

곡	T (8분음표)	N (고유 note) †	화음 수
hibari	1088	23	17
solari	224	34	49
Ravel Pavane	548	**49**	230
Bach Fugue	870	**61**	253

† tie 정규화(8분음표 기준, §4 참고) 적용 후 고유 note (pitch, dur) 쌍 수.

거리 함수별 Algo1 JS

곡	frequency	tonnetz	voice leading	DFT	최적
hibari	0.0344	0.0493	0.0566	0.0213	**DFT**
solari	0.0634	0.0632	0.0775	0.0824	Tonnetz (frequency)
Ravel Pavane	0.0337	0.0415	0.0798	0.0494	**frequency**
Bach Fugue	0.0902	0.0417	0.1242	0.0951	**Tonnetz**

해석

Ravel Pavane: frequency 최적. N=49로 note 다양성이 높은 Ravel에서 빈도 역수 가중치가 가장 효과적이다. Tonnetz는 오히려 JS가 14.8% 악화된다. note 다양성(N=49)이 클수록 빈도 기반 거리 함수(frequency)가 강점을 갖는다는 가설이 지지된다.

Bach Fugue: Tonnetz 최적, voice leading 최악. 푸가는 여러 성부가 같은 주제를 모방 진행하는 **대위법**의 대표 사례이다. Tonnetz가 최적으로 나타난 것은 각 성부가 독립적 선율을 형성하더라도 수직 단면에서 발생하는 화음 연속이 Tonnetz 공간상 구조적으로 유의미함을 시사한다.

거리 함수 패턴 종합:

곡	PC 수	최적 거리	해석
hibari	7 (diatonic)	**DFT**	7음계 스펙트럼 구조 포착 ($k=5$ 성분)
aqua	12 (chromatic)	Tonnetz	화성적 공간 배치
Bach Fugue	12 (chromatic)	Tonnetz	화성적 공간 배치
Ravel Pavane	12	frequency	note 다양성 지배
solari	12	Tonnetz (frequency)	12-PC 구별력 한계, Tonnetz frequency

현재 데이터에서 **hibari만 DFT 최적**이며, 나머지 곡(solari/aqua/Bach/Ravel)은 Tonnetz 또는 frequency가 최적이다. 즉 "거리 함수의 절대 최적"이 있는 것이 아니라 곡의 구조와 목적에 따라 최적이 달라진다. 테스트한 5곡 중 voice leading이 최적인 곡은 없다.

> **참고:** 본 절의 목적은 거리 함수 선택 효과 검증에 있었기 때문에 클래식 대조군 두 곡에 대한 Algorithm 2 적용은 후속 연구 과제로 남긴다.

5.3 위상구조 보존 음악 변주 — 개요

§5.3–§5.6은 "위상구조를 보존하면서 원곡과 다른 음악을 만드는 변주 실험"이다. 본 장의 기본 축은 **Tonnetz 기반**으로 두며, 이는 scale 제약과 화성적 이웃 구조가 Tonnetz와 가장 잘 맞기 때문이다. 이 가정의 타당성은 §5.6.2에서 DFT 전환 실험으로 직접 검증한다.

지금까지의 모든 실험은 원곡의 pitch 분포를 최대한 재현하는 것을 목표로 했다. 이제 방향을 전환하여, **위상구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다. 세 가지 접근을 조합한다: (1) OM 시간 재배치 (§5.4), (2) 화성 제약 기반 note 교체 (§5.5), (3) 두 방법의 결합 (§5.6).

평가 지표 정의.

> **DTW**: Dynamic Time Warping (DTW)은 두 pitch 시퀀스 사이의 거리를 측정한다. 본 연구에서 pitch 시퀀스는 **각 note의 onset 시점을 시간순으로 정렬하여 note당 pitch 값 하나씩을 뽑은 리스트**이다. 즉, 길게 지속되는 음도 짧은 음도 시퀀스에 동일하게 한 번만 포함된다. 두 시퀀스 $x=(x_1,\ldots,x_T)$와 $y=(y_1,\ldots,y_S)$에 대해 DTW는 두 시퀀스의 모든 정렬 경로 중 최소 비용을 갖는 것을 선택한다:

$$\mathrm{DTW}(x, y) = \min_{\text{warpingpath}} \sum_{(i,j)\in\text{path}} |x_i - y_j|$$

여기서 **warping path**는 (1) $(1,1)$에서 (T,S)까지 x, y 각 성분이 단조 증가하며 (역방향 불가), (2) 각 단계에서 x, y 각 성분이 최대 1 증가하는 정렬 경로이다. DTW는 일반 유클리드 거리와 달리 시간 축의 국소적 신축을 허용하여 선율의 전반적인 윤곽(pitch 진행 패턴)을 비교한다. 값이 클수록 두 곡의 선율이 더 많이 다르다.

§5.3–§5.6의 모든 실험에서 Algorithm 1 및 Algorithm 2는 **원곡 hibari의 OM**을 그대로 사용한다. OM의 구조(몇 개의 note로 구성된 어떤 cycle들이 시계열에서 어떤 양상으로 중첩된 채 활성화되는지)는 원곡과 동일하며, 변주는 언제 연주하느냐(§5.4) 또는 어떤 note를 사용하느냐(§5.5)에서만 발생한다.

5.4 OM 시간 재배치

OM의 행(시점)을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 동일한 활성화 패턴이 연속되는 구간을 식별하고, 구간 단위로 순서를 셔플. 구간 내부 순서는 유지. 패턴이 바뀌는 시점을 경계로 삼으므로 구간 길이가 가변적이다. hibari DFT 기준 $T=1088$에서 segment 수가 1012개(평균 길이 1.08)로, 시작/끝 일부를 제외하면 사실상 1-step 구간이 대부분이다.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.
3. **markov_resample** ($\tau=1.0$): 원본 OM의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (§2.10).

평가 지표 보충. transition JS는 두 곡의 note-to-note 전이 분포 간 JS divergence이다.

세 모델의 시간 재배치 반응. FC, LSTM, Transformer는 시간 재배치에 대해 구조적으로 다른 반응을 보인다.

FC: 각 시점 t의 cycle 활성 벡터를 독립 처리하므로, OM 행 순서가 바뀌어도 **pitch 분포는 거의 불변**이다. baseline/segment/block32의 pitch JS가 모두 0.000373 ± 0.000281로 동일했고, markov만 0.001030 ± 0.000087로 미세하게 상승했다. 반면 DTW는 baseline 대비 segment $+47.8\%$, block32 $+30.3\%$, markov $+34.1\%$로 크게 증가해, FC가 "같은 pitch 분포를 유지하면서 선율 시간 순서만 바꾸는" 변주에 구조적으로 적합함을 확인했다.

LSTM: 실측 기준으로 retrain X(약한 재배치)의 세 전략 DTW 변화가 모두 $\leq 0.5\%$였다 (segment $+0.11\%$, block $+0.12\%$, markov $+0.36\%$). retrain O(강한 재배치)에서도 segment는 -1.09%로 제한적이다. pJS는 baseline부터 $0.26 \sim 0.28$로 분포가 붕괴되어 재배치 전략 간 비교 의미가 약하므로 서술을 생략한다.

Transformer: 명시적 PE로 시간 위치를 학습하므로, **PE 유무**가 재배치 효과의 핵심 변수다.

Transformer 결과 (PE 제거 + 재배치)

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 OM으로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE_baseline	0.011	0.128	1.85	—
noPE_segment_shuffle (retrain X)	0.011	0.149	1.87	$+1.0\%$
noPE_markov (retrain X)	0.010	0.138	1.87	$+0.9\%$
noPE+retrain segment_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau=1.0$)	0.185	0.443	2.16	$+18.0\%$

딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.011 \to 0.173$으로 **분포가 붕괴**된다.

약한 재배치 → pitch 분포 유지, 선율 변화 없음

강한 재배치 → pitch 분포 붕괴, 선율 변화

이는 시간 재배치 단독으로는 "pitch 분포 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다.

5.5 화성 제약 조건

위상구조를 보존하면서 note를 교체할 때, 제약 없이 선택하면 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 note 교체의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 consonance score(§2.10)를 penalty로 추가.
3. **interval structure 보존**: 원곡의 interval class vector와 새 note 집합의 ICV 차이를 penalty로 추가.

통합 비용 함수

위 세 제약(scale, consonance, interval)은 동시 적용 가능하다. 전체 비용 함수는

$$\mathrm{cost}(S_{\mathrm{new}}) = \alpha_{\mathrm{note}} \cdot \mathrm{dist_err} + \mathbb{1}[\mathrm{use_diss}] \cdot \beta_{\mathrm{diss}} \cdot \mathrm{cons_score} + \mathbb{1}[\mathrm{use_int}] \cdot \gamma_{\mathrm{icv}} \cdot \mathrm{ICV_diff}$$

이며(본 실험 기본값: $\alpha_{\mathrm{note}} = 0.5$, $\beta_{\mathrm{diss}} = 0.3$, $\gamma_{\mathrm{icv}} = 0.3$), scale 제약은 **후보 pool 자체를 축소**하는 방식으로 작용하므로 cost 수식에서 제외하였다.

$$\mathrm{dist_err}(S_{\mathrm{orig}}, S_{\mathrm{new}}) = \frac{1}{\binom{n}{2}} \sum_{i<j} \left| d(n_i^{\mathrm{orig}}, n_j^{\mathrm{orig}}) - d(n_i^{\mathrm{new}}, n_j^{\mathrm{new}}) \right|$$

$\mathrm{dist_err}$는 **원곡 note 쌍의 위상 거리와 재분배 note 쌍의 위상 거리 사이의 평균 절대 오차**로 정의된다. 값이 0이면 재분배가 모든 쌍 거리를 완전히 보존한 것이고, 클수록 위상구조가 왜곡된 것이다. 단위는 사용한 거리 함수에 의존한다 — Tonnetz는 hop 수, voice_leading은 반음 수, DFT는 Fourier 계수 L^2 norm으로 각기 다르므로, **같은 거리 함수 내에서만 dist_err 값을 비교할 수 있다.** 본 절(§5.5)은 Tonnetz note metric을 기준으로 한다.

$$\mathrm{ICV_diff}(S_{\mathrm{orig}}, S_{\mathrm{new}}) = \|\mathrm{ICV}(S_{\mathrm{orig}}) - \mathrm{ICV}(S_{\mathrm{new}})\|_1 = \sum_{k=1}^{6} |\mathrm{ICV}(S_{\mathrm{orig}})[k] - \mathrm{ICV}(S_{\mathrm{new}})[k]|$$

ICV 차이는 **두 집합의 ICV 벡터 사이의 L^1 노름**으로 정의된다. 값이 0이면 두 note 집합이 정확히 같은 interval class 분포를 갖는 것이다. pitch class 집합의 ICV는 이조(transposition) 하에서 불변이므로, 이 제약은 조성 이동은 허용하되 화성 구조 자체는 보존하는 재분배를 선호하게 한다.

아래 Algorithm 1 결과표의 각 행은 다음 mode에 대응한다:

행 이름	use_scale	use_diss	use_int
baseline	×	×	×
scale_major / minor / penta	○ (pool 축소)	×	×
consonance 단독	×	○	×
interval 단독	×	×	○

Algorithm 1 결과

설정	dist_err	cons_score	scale match	PC 수
baseline (제약 없음)	4.35	0.412	0.70 (C major)	10
scale_major ★	3.52	0.361	1.00 (C major)	7
scale_minor	3.68	0.363	1.00 (E major)	7
scale_penta	4.32	0.213	1.00 (C major)	5
consonance 단독	4.35	0.412	0.70	10
interval 단독	4.35	0.437	0.64	11

1000개 후보 집합에서 비용 함수를 최소화하는 **결정론적 조합 최적화** 결과이므로, seed 반복 실험이 원리적으로 불필요하다 (각 설정마다 단일 해가 유일하게 결정됨). 따라서 N, std 표기를 생략한다.

핵심 발견:

1. **scale_major가 가장 효과적**: dist_err가 baseline $4.35 \rightarrow 3.52$로 19% 감소하면서 동시에 scale match 1.00, consonance score $0.412 \rightarrow 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta에서 가장 낮은 consonance score**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. consonance score가 거리 보존 제약에 비해 너무 약한 것으로 추정된다. ($\alpha_{\text{note}}, \beta_{\text{diss}}$) = ($0.5$, 0.3)은 두 항의 스케일 균형을 위한 휴리스틱 기본값이며, grid search는 후속 과제이다.

Algorithm 2 (Transformer) 결과

(화성 제약 실험에서는 Algorithm 2로 **Transformer**를 사용한다. FC는 §5.4에서 시간 재배치에 무관함이 확인되었으며, FC의 DL 성능 비교는 §5.8.2에서 DFT 기반 실험으로 별도 수행된다.)

original 행은 재분배를 적용하지 않은 원곡 hibari의 OM을 그대로 Transformer에 학습시킨 baseline이다. 본 §5.5 실험 전체에서 **이진 OM** 을 사용한다 (continuous OM 실험은 §5.6에서 별도 진행).

설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val_loss
original	0.009	1.80	—	0.524
baseline (제약 없음)	0.600	3.46	0.007	0.497
scale_major ★	0.097	2.35	0.003	0.492
scale_penta	0.259	3.37	0.009	0.487

$N=1$ (단일 seed 학습). Transformer 학습은 원리적으로 seed에 의존하여 seed를 바꿔가며 5회 이상 실험하는 것이 바람직하다.

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (JS 최댓값의 14.0%), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽하게 학습)으로, **위상 보존 + 정량화 가능한 차이 + 화성적 일관성**의 균형이 가장 좋다. —

위상 보존: §5.5 실험 전체에서 **원곡 hibari의 이진 OM을 그대로 사용**한다. 추가로 dist_err 3.52 (Algorithm 1 표의 scale_major 행)는 pair-wise 거리 구조도 baseline 대비 19% 개선되어 보존됨을 수치적으로 뒷받침한다.

정량화 가능한 차이: DTW 2.35 ($+31\%$) 및 pJS 0.097 ($\ln 2$의 14%)이 근거. 특히 DTW는 선율 윤곽의 차이를, pJS는 pitch 분포의 차이를 독립적으로 수량화한다 — 이 두 값이 동시에 양의 방향으로 변하므로 "원곡과 다름"을 두 축에서 수치로 주장할 수 있다.

화성적 일관성: scale_major 제약으로 scale match $=1.00$ (원곡 hibari의 조성과 일치), 그리고 Algorithm 1 표의 consonance score 0.361 (baseline 대비 12% 개선)이 근거이다.

보강 실험 (DFT): FC/LSTM도 같은 설정에서 확인했다.

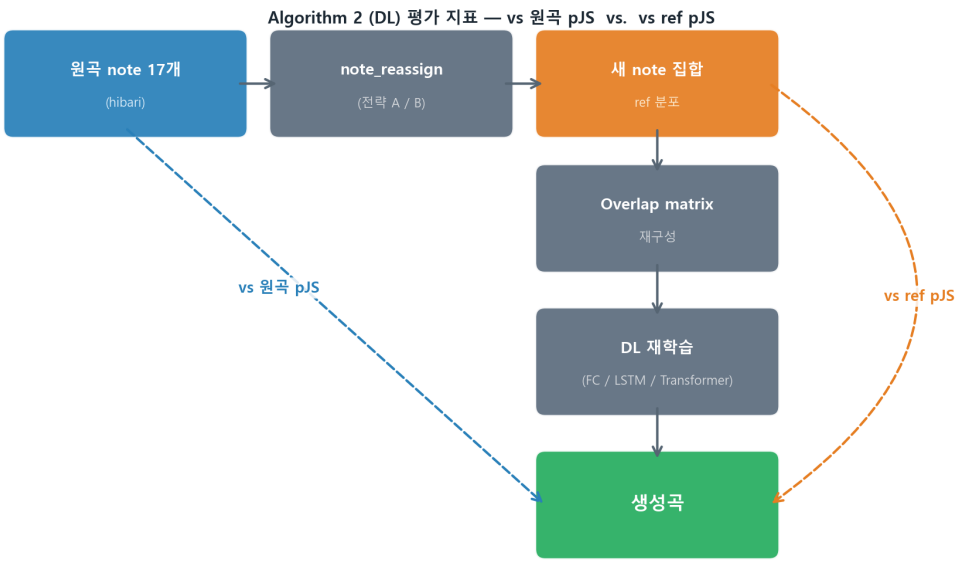
모델 (DFT, scale_major, N=5)	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val_loss
FC	0.3224 ± 0.0070	4.7397 ± 0.0692	0.0051 ± 0.0026	0.0393 ± 0.0070
LSTM	0.5211 ± 0.0168	6.1632 ± 0.1860	0.2832 ± 0.0012	0.3972 ± 0.0075

$N = 5$ 독립 seed 재학습의 mean $\pm$ std.

LSTM은 화성 제약 학습에서도 열세다. Transformer의 DFT 결과(vs 원곡 pJS $= 0.3133$)는 §5.6.2 통합 비교에서 해석한다. full 원고에는 original/baseline/scale_major/scale_penta 전체 행을 mean ± std로 제시한다.

> **FC vs ref pJS.** DFT scale_major 조건에서 FC의 vs ref pJS (0.0051 ± 0.0026)는 Transformer의 동 조건 vs ref pJS (0.015156) 대비 약 **66% 낮다** — FC가 재분배된 note 분포를 훨씬 정밀하게 학습한다는 증거. §5.4에서 확인된 FC의 pitch 분포 고정 특성(시점 독립 처리)이 scale 제약 학습에서도 유효하다.

> **note 재분배 범위.** §5.5 실험에서 note 재분배의 pitch 범위는 Algorithm 1 및 Transformer 모두 **wide (48–84)** 기준이며, baseline (원곡 그대로, 재분배 없음)은 재분배 적용 전 원곡 OM을 그대로 학습한 것이다. hibari의 pitch 범위는 52–81이다.



5.6 화성 제약 + 시간 재배치 + 연속값 OM — 통합 실험

5.6.1 Tonnetz 기반 통합 실험 ($N = 10$)

설정 (Tonnetz)	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	scale match
major_block32	0.2696 ± 0.111	3.620 ± 0.818	0.00710 ± 0.00308	1.00

major_block32는 재분배 분포 학습(ref pJS)과 조성 일치(scale match 1.0)를 동시에 만족한다.

5.6.2 DFT 전환 탐색

비교 설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS
Tonnetz Transformer major_block32	0.2696 ± 0.111	3.620 ± 0.818	0.00710 ± 0.00308
DFT Transformer major_block32	0.2689 ± 0.0736	3.105 ± 0.428	0.01622 ± 0.00267

§5.6.1에서 거리 함수를 Tonnetz에서 DFT로 변경 시 ref pJS가 **약 2.28배** 악화된다

5.6.3 메타 통찰 — 거리 함수 × 음악적 목적

위상구조 재현(원곡 재현·모듈 탐색) 목적에서는 DFT가 강점.

화성 정합성 성취(scale 제약 변주·조성 유지) 목적에서는 Tonnetz가 강점.

FC의 pitch 분포 유지 특성. FC는 OM 재배치 하에서 pitch 분포를 거의 고정한 채 DTW를 $+30 \sim +48\%$ 변화시켜, "pitch 유지 + 선율 순서 변화" 변주에 특히 적합했다. 그럼에도 §5.6 전체 최적이 Tonnetz-Transformer로 남는 이유는 다음과 같다: **OM 재배치**보다 **note를 고르는 단계**에서 작용하는 scale 제약이 더 큰 효과를 낸다. 특히 Tonnetz 기반 실험에서 scale 제약의 이득이 DFT 기반 대비 약 2.28배(§5.6.2) 크게 나타났는데, 이는 Tonnetz가 화성적 근접성을 직접 인코딩하므로 scale 제약과 상호보완적으로 작동하기 때문이다.

따라서 단일한 거리 함수가 보편적으로 최적이라기보다, **거리 함수를 음악적 목적에 보다 정합적으로** 설계해야 함을 의미한다. 이는 §5.9의 complex 모드 결과(DFT 악화)와도 일치한다.

5.7 DFT Hybrid의 α grid search — 실험 결과

DFT hybrid 거리 $d_{\text{hybrid}} = \alpha \cdot d_{\text{freq}} + (1-\alpha) \cdot d_{\text{DFT}}$ 에서 $\alpha \in \{0.0, 0.1, 0.25, 0.3, 0.5, 0.7, 1.0\}$, $N = 20$ 반복 (Algorithm 1). d_{DFT} 내부 파라미터 $(w_o, w_d) = (0.3, 1.0)$ 는 §4.1a / §4.1b DFT 조건 grid search로 확정된 고정값이며, 본 표의 모든 행에 공통 적용된다.

α	K	JS (mean ± std)	비고
0.0 (pure DFT)	**1**	0.0728 ± 0.00432	K=1, degenerate
0.1	13	0.01602 ± 0.00204	
0.25	**14**	0.01593 ± 0.00181	**최적**
0.3	16	0.02025 ± 0.00134	
0.5	19	0.01691 ± 0.00143	
0.7	24	0.03140 ± 0.00270	
1.0 (pure freq)	**1**	0.03386 ± 0.00186	K=1, degenerate

DFT hybrid에서는 **양 끝점 모두 degenerate** ($K = 1$). $\alpha = 0.25$가 최적 ($K = 14$). 이 결과는 §5.8의 per-cycle τ_c 실험에서도 재확인된다 — $\alpha = 0.25$ 조건의 per-cycle τ_c가 JS $= 0.00902$으로 최저를 기록하며, $\alpha = 0.25$**가 이진 OM과 per-cycle** τ_c **양쪽 모두에서 최적임이 이중으로 검증된다**.

5.8 연속값 OM의 정교화 — 실험 결과

DFT 기준 §4.2 결과는 **이진 OM이 최적**(JS 0.0157)이며, continuous→단일 τ 이진화는 $\tau = 0.3/0.5/0.7$에서 각각 $0.0360/0.0507/0.0449$로 모두 열세였다. 본 절은 이 단일 임계값 한계를 넘어, 두 가지 정교화 실험을 추가 수행한 결과를 보고한다.

5.8.0 Cycle별 활성화 프로파일의 다양성

Per-cycle τ 실험에 앞서, 왜 cycle마다 서로 다른 임계값이 필요한지를 직관적으로 설명한다.

각 cycle의 연속 활성화 값 $O_{\text{cont}}[t, c] \in [0, 1]$은 "이 cycle을 구성하는 note들이 시점 t에서 얼마나 많이, 얼마나 드물게 울리는가"를 나타낸다. 이 값은 cycle의 음악적 역할에 따라 극적으로 다른 분포를 보인다.

Cycle A형 (지속 활성형). 두 악기 모두에서 지속적으로 반복 등장하는 음들로 구성된 cycle은 거의 전 구간에서 약하게 활성화되며 O_{cont} 값이 안정적으로 낮다 (예: 0.15–0.30). 균일 임계값 $\tau = 0.35$를 쓰면 이 cycle은 대부분의 시점에서 비활성으로 처리되어 지속적 선율 배경의 역할이 무시된다.

Cycle B형 (색채음형). 원곡에서 드물게 등장하는 note들이 포함된 cycle은 해당 note들이 나타나는 구간에서 $O_{\text{cont}} \approx 0.6$ – 0.9로 급상승한다고 관찰된다. 더 높은 $\tau = 0.60$–0.70을 쓰면 "확실히 의도된 구간"만 활성화되어 색채가 더 선명해진다.

즉, τ를 cycle마다 다르게 설정해야 각 cycle의 음악적 기능이 제대로 표현된다.

5.8.1 Per-cycle 임계값 최적화

방법 (공통). Cycle c를 인덱스 순서대로 순회하며, 나머지 cycle의 τ를 고정한 채 τ_c를 $\{0.1, 0.2, 0.3, 0.35, 0.4, 0.5, 0.6, 0.7\}$ 중 JS 최소값으로 결정하는 **1-pass greedy coordinate descent**를 수행한다. 초기값은 전역 $\tau = 0.35$이며, 순회 순서와 seed에 의존하므로 전역 최적을 보장하지 않는다. 전역 최적 탐색은 후속 과제로 남긴다.

Phase 1 — $\alpha = 0.5$ 기준 결과 ($N = 20$, DFT baseline, $\alpha = 0.5$, $K = 19$) :

설정	JS (mean ± std)	per-cycle 대비	Welch p
§4.2 Binary OM ★	0.0157 ± 0.0018	$+5.5\%$	—
§4.2 τ=0.3 이진화	0.0360 ± 0.0029	$+58.6\%$	—
per-cycle τ_c (α=0.5)	0.01489 ± 0.00143	—	2.48×10^{-26} (vs τ=0.3 이진화)

최적 τ_c 분포 ($K = 19$): $\tau = 0.7$이 5개 (26.3%), $\tau = 0.1$이 3개 (15.8%). 중앙값 $\tau = 0.4$.

Phase 2 — $\alpha = 0.25$ 기준: §5.7 DFT hybrid grid search 최적 $\alpha = 0.25$ ($K = 14$) 조건에서 per-cycle τ_c 재탐색.

설정	α	K	JS (mean ± std)	Welch p
이진 OM (§5.7)	0.25	14	0.01593 ± 0.00181	—
per-cycle τ_c (기준, 비교)	0.5	19	0.01489 ± 0.00143	—
per-cycle τ_c ★ 신기록	0.25	14	0.00902 ± 0.00170	4.94×10^{-11}

최적 τ_c 프로파일 ($K = 14$): $[0.7, 0.7, 0.3, 0.7, 0.7, 0.4, 0.1, 0.3, 0.4, 0.1, 0.5, 0.35, 0.4, 0.3]$.

$\alpha = 0.25$ 이중 검증 — Algo1 본 연구 최저 갱신. $\alpha = 0.25$에서 cycle별 독립 임계값 최적화가 추가 개선을 달성한다. Per-cycle τ_c ($\alpha = 0.25$, $K = 14$) JS $= 0.00902 \pm 0.00170$가 **Algorithm 1 기준 본 연구 전체 최저**이다.

α-grid 결과 ($N = 20$, DFT, per-cycle τ_c). $\alpha = 0.25$ 조건이 $\alpha = 0.50$ 대비 JS 71.48% 우세하다 (Welch $p = 1.44 \times 10^{-15}$).

α	JS (mean ± std, $N = 20$)	$\alpha = 0.50$ 대비
0.25	0.00902 ± 0.00170 ★	-41.7%
0.30	0.01032 ± 0.00121	-33.3%
0.50	0.01547 ± 0.00125	—

5.8.2 연속값 OM을 직접 받아들이는 Algorithm 2

per-cycle τ는 본 절에 적용되지 않는다. Algorithm 2(DL)에 대해선 DFT 기반 연속값 OM을 이진화하지 않고 있는 그대로 넣는 것이 가장 효과적임을 보았다. 따라서 본 절은 §5.8 "연속값 OM 정교화" 맥락에서 핵심 발견(FC-cont 우위, §4.3)을 재제시한다(per-cycle τ_c는 §5.8.1 Algorithm 1 한정).

Phase 1 — $\alpha = 0.5$ baseline ($N = 10$, $K = 19$).

모델	입력	JS (mean ± std)	val_loss	개선율
FC	Binary	0.00217 ± 0.000565	0.3395	—
FC	**Continuous**	0.000348 ± 0.000149	0.0232	$+84.0\%$

Transformer	Binary	0.00251 ± 0.000569	0.836	—
Transformer	Continuous	0.000818 ± 0.000255	0.152	$+67.4\%$
LSTM	Binary	0.233 ± 0.0289	0.408	—
LSTM	Continuous	0.170 ± 0.0272	0.395	$+27.3\%$

FC-cont vs Transformer-cont : Welch $p = 1.66 \times 10^{-4}$ (FC 유의 우위).

Phase 2 — $\alpha = 0.25$ 재실험 ($N = 10$, $K = 14$).

모델	입력	α	K	JS (mean ± std)
FC	Continuous	0.5 (Phase 1)	19	0.000348 ± 0.000149 ★
FC	Continuous	0.25 (Phase 2)	14	0.00057 ± 0.00046

§5.8.1에서 $\alpha = 0.25$가 Algo1 per-cycle τ_c 최적으로 확인됨에 따라, Algo2 FC-cont에서도 동일 α로 재검증하였다. Phase 1 vs Phase 2 Welch $p = 0.168$. 관측 수치상으로는 $\alpha = 0.5$가 더 낮지만(0.000348 vs 0.00057), 이 차이는 통계적으로 유의하지 않다. K 감소 (19 → 14)로 FC의 cell-wise 학습 신호가 줄어든 것이 가능한 원인으로 예상되며, **Algo2 최저는 $\alpha = 0.5$, FC + continuous로 유지**한다. 이는 §5.8.1 Algo1 최적($\alpha = 0.25$)과는 다른 하이퍼파라미터 구성에서 Algo2 최적이 형성됨을 뜻하며, 두 알고리즘의 학습 신호 특성 차이를 반영한다.

5.9 Simul 혼합 모드

Complex weight:

$$W_{\text{complex}}(r_c) = W_{\text{timeflow}} + r_c \cdot W_{\text{simul}}$$

아래 표는 거리 함수 DFT, per-cycle τ_c 적용 OM 조건에서 timeflow 기준 §5.8.1 결과 대비 complex 모드 d_{freq}($\alpha = 0.25$)의 영향을 정리한다. 모든 complex 행은 $\alpha = 0.25$로 통일하였으며, Tonnetz 행은 추가로 $\omega_o = 0.0$ 조건이다.

실험	α	r_c	Algo 1 JS	§5.8.1 대비
§5.8.1 ★ ($\alpha = 0.25$)	0.25	—	0.00902 ± 0.00170	—
complex (DFT)	0.25	0.1	0.0440 ± 0.0010	$+388\%$ (악화), $p = 4.74 \times 10^{-39}$
complex (DFT)	0.25	0.3	0.0657 ± 0.0015	$+628\%$ (악화), $p = 1.12 \times 10^{-48}$
complex (Tonnetz, $\omega_o = 0.0$)	0.25	0.1 ★	0.0183 ± 0.0009	(Tonnetz 한정 효과)

Tonnetz 조건에서 r_c 최적값 검증 결과 ($N = 20$): $r_c = 0.1$ (JS $= 0.0183$) vs $r_c = 0.3$ (JS $= 0.0214$) vs $\alpha = 0.5$ 대조 (JS $= 0.0218$) — $r_c = 0.1$이 p<0.001 유의 최적.

결론: DFT baseline에서는 timeflow + per-cycle τ_c 조합이 최적이며, complex 혼합 모드는 DFT에서 유의하게 악화된다. Complex 모드는 **Tonnetz 한정으로만 유효**하다 (§5.6.3 메타 통찰 근거). 따라서 hibari의 최종 설정에서는 timeflow 모드를 유지한다.